

Full Length Article

LogSqueeze: Real-time APT log compression via spatiotemporal feature fusion and multi-stage provenance graph evolution

Tiantian Zhu^{a,b,1}, Yiqian Yang^a, Zhengqiu Weng^{c,1}, Haoifei Sun^a, Mingjun Ma^a, Zhizhong Ma^{c,*}, Suyu Zhang^{d,*}^a Zhejiang University of Technology, China^b Taizhou Research Institute, Zhejiang University of Technology, China^c Wenzhou University of Technology, China^d Wenzhou Polytechnic University, China

ARTICLE INFO

Keywords:

Advanced persistent threat spatiotemporal analysis

Provenance graphs adaptive scheme

Graph-based knowledge

ABSTRACT

Advanced Persistent Threat (APT) attacks have emerged as one of the most critical threats in cyberspace, characterized by their persistent stealth and dynamic evolution. System logs, as critical records of system activities and anomalies, form the foundation for APT detection and forensics. However, the rapid growth of log data has led to prohibitively high costs for real-time attack detection, making efficient compression of streaming logs critical. Current data compression methods face limitations in scenario adaptability, key information preservation, and computational overhead, failing to address multi-scenario APT defense requirements. To address the above problems, in this paper we propose LogSqueeze, a real-time APT log compression system by using spatiotemporal feature fusion and multi-stage provenance graph evolution. LogSqueeze 1) compress system logs via spatiotemporal analysis (window aggregation, differential/dictionary coding), 2) retain attack-critical data using evolving multi-level provenance graphs, and 3) optimize adaptively with dynamic compression tuning. Experimental results demonstrate that LogSqueeze can significantly enhance data processing speed in the real-time detection scenario, quickly locate potential attack signs. While ensuring a detection accuracy rate of 98.2%, it increases the real-time compression rate by more than 30%.

1. Introduction

Currently, the network security situation is becoming increasingly severe, in which Advanced Persistent Threat (APT) has become an important weapon to destroy critical infrastructure such as defense and military industries, energy, and communications due to its hidden, targeted, and persistent characteristics. According to the 2024 Global Advanced Persistent Threat Research Report published by 360 Digital Security (Global Advanced Persistent Threat (APT) Research Report, 2025), global network security vendors and organizations have cumulatively released more than 730 APT reports, involving 124 APT organizations, of which 41 APT organizations were disclosed for the first time.

System logs (e.g., ETW for Windows (Author Unknown, 2025), and audit for Linux (Author Unknown, 2025)) serve as evidence for recording system activities, detailing low-level interaction information

(e.g., processes, files, network connections, etc.) and building provenance graphs (Xie et al., 2012; Xie et al., 2011; Ahmad et al., 2018). They provide important support for the detection of APT attacks (Hossain et al., 2017; Tang et al., 2018; Milajerdi et al., 2019). According to the CrowdStrike 2025 Global Threat Report (CrowdStrike, 2025), the breakout time has reached an all-time low (48 min on average) in the past year, which means that if analysts are not able to detect and deal with the threat in real-time, the attacker can cause further harm by moving laterally in a short period of time. However, in real enterprise environments, the amount of system logs tends to explode over time (e.g., a large financial institution generated >50,000 events in just 2 min (Qianxin, 2025)). To this end, log compression techniques are indispensable to correlate long APT attack data and ensure real-time detection. In streaming real-time APT attack detection scenarios, an ideal data compression strategy should be able to quickly process a large number of kernel logs under the premise of ensuring

* Corresponding authors.

E-mail addresses: mazhizhong@wzut.edu.cn (Z. Ma), zsy8311@163.com (S. Zhang).¹ Tiantian Zhu and Zhengqiu Weng share dual first authorship.<https://doi.org/10.1016/j.cose.2026.105169>

Received 19 June 2025; Received in revised form 9 May 2026; Accepted 13 September 2026

Available online 14 September 2026

0167-4048/© 2026 Elsevier Ltd. All rights are reserved, including those for text and data mining, AI training, and similar technologies.

detection accuracy to realize real-time attack response. Although existing mainstream compression methods have made some progress, the following problems still exist:

- **Lack of effectiveness (C1).** Existing methods may mistakenly delete key attack information or omit attack-irrelevant information during the compression process, making it difficult for analysts to effectively identify and locate the full picture of behaviors in an APT attack. LogGC (Lee et al., 2013) deals with isolated temporary files through garbage collection, but may misjudge the event correlation in complex causality. Lee et al. (2013) simplifies causality by enforcing partitioning, but may lose key causal connections in dynamic data usage scenarios. CPR (Xu et al., 2016) retains key information through information flow retention techniques but is ineffective when dealing with densely interleaved events. PCAR (Xu et al., 2016) aggressively reduces data redundancy and frees up storage space, but significantly degrades anomaly detection performance and affects attack data retention. RapSheet (Hassan et al., 2020) removes non-attack-related entities but may misjudge entity-attack relevance, leading to misdeletion of useful information.
- **Lack of real-time (C2).** The real-time requirement for APT attack detection is extremely high, and the latency and resource consumption issues of existing methods may result in security personnel not being able to detect and respond to attacks in a timely manner. Performing structural compression using subgraphs (e.g., StarZIP (Dolgorsuren et al., 2019) and GraphZIP (Rossi and Zhou, 2018)) maintains a high compression rate but has a high computational overhead in the preprocessing stage, which makes it unsuitable for real-time data processing. Pattern extraction techniques (Shabani et al., 2023; Jamshidi et al., 2020), which provide a high accuracy of compression rate but have a long processing time, make it difficult to be applied in real-time environments. While machine learning based compression methods (e.g., LEONARD (Ding et al., 2023) and DeepZip (Goyal et al., 2018)) are able to retain global log semantic information, but the difficulty of training models (Radford et al., 2019), resource consumption, and accurate extraction of log redundancy lead to poor compression results. Related methods (Hassan et al., 2020; Hossain et al., 2018; Hossain et al., 2018) require the entire provenance graph as input in the compression process and cannot handle streaming data.
- **Lack of dynamic adaptation (C3).** APT attackers may use camouflage and bypass to interfere with log analysis, and existing data compression techniques are difficult to consistently apply in dynamically changing network environments. Nodoe (Hassan et al., 2019) uses probabilistic transfer matrices to identify and retain suspicious system activities, but the boundaries between benign and suspicious activities are difficult to clearly delineate and it is easy to mistakenly delete key semantic information. ProvG-Searcher (Altinisk et al., 2023) prunes redundant event chains using expert-guided activity labeling, but incurs scalability bottlenecks due to manual effort under dynamic enterprise conditions. GS (Zhu et al., 2021) achieves semantics-preserving compression through node/event equivalence analysis, but suffers from suboptimal efficiency due to temporal-feature-only modeling.

To address these limitations, we propose LogSqueeze, a real-time APT log compression system that integrates spatiotemporal feature fusion and multi-stage provenance graph evolution for optimized attack-relevant data retention. Firstly, LogSqueeze analyzes log spatiotemporal features, performing online log template extraction and time/space-aware compression to output lightweight data for the construction of multi-level provenance graphs. This significantly improves compression effectiveness (C1) and real-time responsiveness (C2). Secondly, LogSqueeze then employs dynamic multi-level provenance graph evolution to prioritize retention of attack-relevant critical data, enabling accurate attack path identification and key information extraction in complex

scenarios. This approach maintains compression effectiveness and real-time performance while significantly improving adaptability to dynamic environments (C3). Finally, through the adaptive compression and dynamic adjustment strategy, LogSqueeze is able to automatically adjust the compression parameters according to changes in data flow and system load, further optimizing compression efficiency and real-time performance. In summary, the contributions of this paper are as follows:

- Unlike existing batch-processing or semantic-only methods, we propose a three-stage streaming pipeline that achieves a processing throughput of 52 K events/s, significantly exceeding the typical data generation rate (430 events/s). This fusion of online template extraction and adaptive windowing delivers a compression ratio up to $12.6 \times$, outperforming SOTA approaches like CPR and GS by over 30% while maintaining real-time responsiveness.
- We introduce a multi-stage provenance graph evolution mechanism that reduces memory complexity from $O(|E| + |V|)$ to $O(|V|)$ by maintaining only active entity states. Furthermore, by integrating a root node optimization strategy, we improve the LSTM's longrange dependency capture, achieving an average detection accuracy of 98.2.
- We compare LogSqueeze with SOTA methods (i.e., CPR, GS, and F-DPR) on the U.S. Defense Advanced Research Projects Agency (DARPA) Transparent Computing (TC) (Darap3, 2023) E3 dataset, the Darpa TC E4 dataset, and real-world deployed cyber range datasets in our laboratory environments. Experimental results demonstrate that our method achieves up to $12.6 \times$ log compression ratio, significantly outperforming SOTA approaches. LogSqueeze delivers over 30% higher compression rates while maintaining 98.2% average detection accuracy.

2. Related work

In this section, we systematically review state-of-the-art data compression methods, benchmarking their effectiveness, real-time processing capability, and environmental adaptability to contextualize our methodological advances.

2.1. Effectiveness

In terms of the effectiveness of data compression, existing research has focused on compression methods based on redundant and semantic information. Data compression methods based on redundant information reduce the data size by removing duplicate or irrelevant information from the logs, for example, LogGC (Lee et al., 2013) removes isolated temporary file events through a garbage collection mechanism. NodeMerge (Tang et al., 2018) utilizes the property of applications that load global read-only files for merging to reduce redundancy. Ma et al. uses the KCAL framework (Ma et al., 2018) to remove redundant causal events with kernel-level caching. However, these approaches may lose critical information or misjudge causality when dealing with complex causal and dynamic data scenarios (e.g., NodeMerge (Tang et al., 2018) may destroy the timing integrity of the process call chain when deleting duplicate nodes, resulting in a 19% decrease in the accuracy of covert channel attack detection). For the retention of critical information, Lee et al. (2013) simplifies causality by executing binary partitioning (Lee et al., 2013), which identifies and records critical events such as system calls and file operations related to the attack by analyzing binary code during program execution, but critical causal links can be lost in dynamic data usage scenarios. CPR (Xu et al., 2016) retains critical information through information flow retention techniques and subsequently PCAR (Xu et al., 2016) is an innovative extension based on the CPR system, which can monitor the system call frequency of processes in real-time, reduce data redundancy, and free up storage space. However, these methods significantly degrade the anomaly detection performance and affect the attack data retention. RapSheet (Hassan et al., 2020)

removes non-attack related entities by matching alert events, but it may incorrectly judge the entity's relevance to the attack and result in the erroneous deletion of useful information (Symantec, 2023; Red Canary, 2024).

Unlike existing methods that may mistakenly delete key attack information or omit attack-irrelevant information during the compression process, LogSqueeze strips off redundant information layer-by-layer by means of online template extraction, time-window aggregation, online differential coding, and dictionary coding techniques, and prioritizes retaining the key data that are highly relevant to the attack, so that efficient compression can be achieved based on the assurance of data validity.

2.2. Real-time performance

Gandhi et al. (2023) proposed a rethinking of system audit architectures to ensure synchronous log availability and high event coverage, addressing the latency gap between event occurrence and analysis. StarZIP (Dolgorsuren et al., 2019) performs structural compression by identifying and reordering the set of star subgraphs, using compression coding techniques to process the graph step by step to perform structural compression, creating a graph streaming object with the help of snapshots and converting an undirected graph into a directed graph. GraphZIP (Rossi and Zhou, 2018) employs clustered subgraph decomposition for streaming graph compression, utilizing sparse matrix representations to optimize disk I/O efficiency. Although the above methods have the advantage of a high compression rate, the computational overhead in the preprocessing stage is large and is not suitable for real-time data processing. Xie et al. (2012), Xie et al. (2013) and Xie et al. (2011) proposed a network graph compression and dictionary encoding scheme for provenance graphs, and Ahmad et al. (2018) extended it to stream processing scenarios. Pattern extraction techniques (Shabani et al., 2023; Jamshidi et al., 2020), provide high accuracy compression rates, but the long processing time makes it difficult to apply in real-time environments. Machine learning based compression methods (e.g., LEONARD (Ding et al., 2023) and DeepZip (Goyal et al., 2018)) automatically learn the distribution pattern of elements from a large amount of data to generate efficient encoding strategies, which are capable of preserving global log semantic information but difficult to train models, consume large amount of resources, and difficult to accurately extract the log redundancy, which leads to poor compression results. DPR (Hossain et al. (Hossain et al., 2018)) retains key forensic information by analyzing node dependencies during intrusions. Its extension F-DPR (Hossain et al., 2018) simplifies provenance graphs through selective event pruning—discarding non-essential streaming events that do not affect ancestor/successor traversal paths. However, this approach requires complete graph ingestion, rendering it incompatible with streaming data processing.

Different from prior approaches, LogSqueeze dynamically adapts to workload fluctuations through runtime-optimized compression strategies. This ensures sustained processing throughput that exceeds data ingestion rates, thereby satisfying stringent real-time monitoring and threat mitigation requirements.

2.3. Latency and actionability

To further justify the real-time nature of LogSqueeze, we evaluate the end-to-end detection latency across the processing pipeline. As shown in our experiments, the cumulative delay—comprising spatiotemporal compression, incremental graph construction, and LSTM inference—is consistently maintained at the millisecond level. Specifically, the lightweight compression stage (template extraction and windowing) introduces negligible per-event delay, while the incremental graph evolution avoids the high latency associated with full-graph reconstruction.

When contextualized against the 48-minute average break-out time

reported in modern APT scenarios, a millisecond-level detection delay provides a significant buffer for security analysts to take action. Since the processing throughput (52 K events/s) far exceeds the ingestion rate (430 events/s), the system avoids queueing delays, ensuring that potential attack signs are localized and alerted almost instantaneously upon log arrival.

2.4. Dynamic adaptability

Nodoze (Hassan et al., 2019) utilizes a probabilistic transfer matrix to identify and retain potentially suspicious system activities while removing normal benign activities from the provenance graph. ProvG-Searcher (Altinisik et al., 2023) removes the similar chain of events based on advanced activity labeling with expert knowledge. However, current methods predominantly integrate anomaly detection principles into data compression frameworks. The inherent challenge of precisely demarcating benign and malicious activities frequently results in inadvertent loss of critical semantic features, consequently compromising the training efficacy and predictive accuracy of threat detection models. GS (Zhu et al., 2021), a lightweight data compression algorithm based on retaining global semantic information, is able to maintain the generality of compression by judging the semantics of the source node and equivalent events, that is, when the semantics of the source node remains unchanged, it deletes duplicate events from the source node that play the same role in the target node. However, considering the temporal characteristics of the events only leads to poor compression effect and does not fully consider the dynamics of the semantic change process.

In terms of detection methodology, Yang et al. (2023) introduced PROGRAPHER, an anomaly detection system based on provenance graph embedding, leveraging structural information for robust identification. Similarly, Wang et al. (2025) proposed ProvGOutLiner, a lightweight detection method focusing on process behavior features to achieve efficiency. LogSqueeze differs by integrating spatiotemporal compression prior to embedding, ensuring that the graph scale remains manageable for long-term monitoring without losing the critical behavioral semantics highlighted by these recent works.

In contrast, LogSqueeze employs a multi-stage provenance graph to dynamically model causal relationships between entities. By integrating Node2Vec for structural embedding and LSTM for temporal pattern learning, LogSqueeze achieves: 1) adaptive operation in dynamic host environments, 2) robust capture of long-range attack dependencies, and 3) improved anomaly detection performance under complex attack scenarios.

3. Background

3.1. Provenance graph

In cyber security research, the provenance graphs (Li et al., 2021) serve as a fundamental technical framework for modeling system interactions. By preserving causal-temporal relationships from log data, they transform discrete system events into behavior chains with contextual semantics, thereby enabling attack behavior visualization through topological mapping. Fig. 1 illustrates the provenance graph of the ccleaner attack, where the nodes denote the entity of the system (e.g., processes, files, and sockets), the edges denote causal relationships between the entities, and the timestamps on the edges record the sequence in which the operations occur. Starting from timestamp 1, SrcSinkObj performs a write operation to /home/admin/ccleaner, and then firefox establishes a connection to the network socket IP:405F19d5:80; the ccleaner process is loaded and executed, which performs write operations to multiple system files (e.g., /home/admin/9006492568, /etc/localtime, etc.) to read and write operations, and at the same time create the dbus-launch process and load the library files; dbus-daemon further loads the library files and reads the system

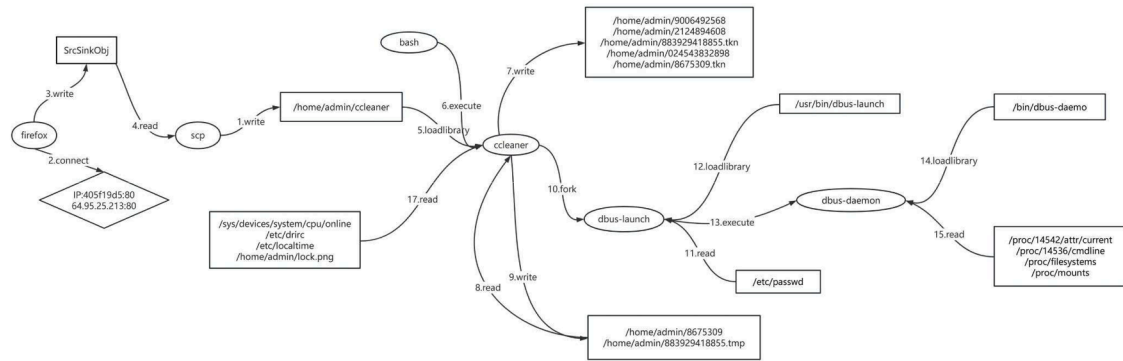


Fig. 1. The attack provenance graph of ccleaner.

configurations, and ultimately completes the lateral movement of the attack and elevation of privileges through cross-process interactions. The provenance graph accurately depicts the complete technical chain of the attacker from initial invasion to target achievement by recording the causal relationship and timestamp between entities, providing a credible behavioral baseline for security analysis.

3.2. Graph embedding technology

Node2Vec is a graph embedding method mainly used to learn low-dimensional vector representations of nodes in graphs to support subsequent machine learning tasks. It captures local and global structural information on the graph by introducing a flexible randomized wandering strategy.

Node2Vec uses a biased stochastic wandering that controls the direction of the wandering through two parameters p and q . The parameter p controls the probability of returning to the previous node and q controls the probability of exploring away from the current node. When p is large, the random walk is less likely to return to the previous node and prefers to explore outward; when p is small, the random walk is more likely to return to the previous node and focus on the local structure. When q is large, the random walk is more likely to wander in the immediate neighborhood of the current node, capturing the local structure; when q is small, the random walk is more likely to explore areas away

from the current node, capturing the global structure.

After performing a biased random walk to generate a sequence of nodes, Word2Vec Skip-Gram model is used to learn the vector representation of the nodes. Generating a sequence of nodes through this biased random walk and then using the Skip-Gram model to learn the vector representation of the nodes allows Node2Vec to flexibly capture the structural information of the graph according to different task requirements.

4. System design

In view of the hidden nature of APT attacks in complex networks of large enterprises, the rapid processing of massive log data becomes the key to detect abnormal behaviors in real-time and respond to them in a timely manner. System design in Fig. 2:

Firstly, we collect and transmit streaming log data, then perform spatiotemporal characteristic analysis for compression. It involves on-line log template extraction, time-window aggregation, online differential coding, and dictionary coding to progressively eliminate redundant information, ultimately outputting compressed data. Then, the multi-stage provenance graph evolution technique is used to optimize storage and maintain graph structural integrity through incremental storage and recursive embedding strategies. Finally, anomaly detection is performed by combining the detection framework of

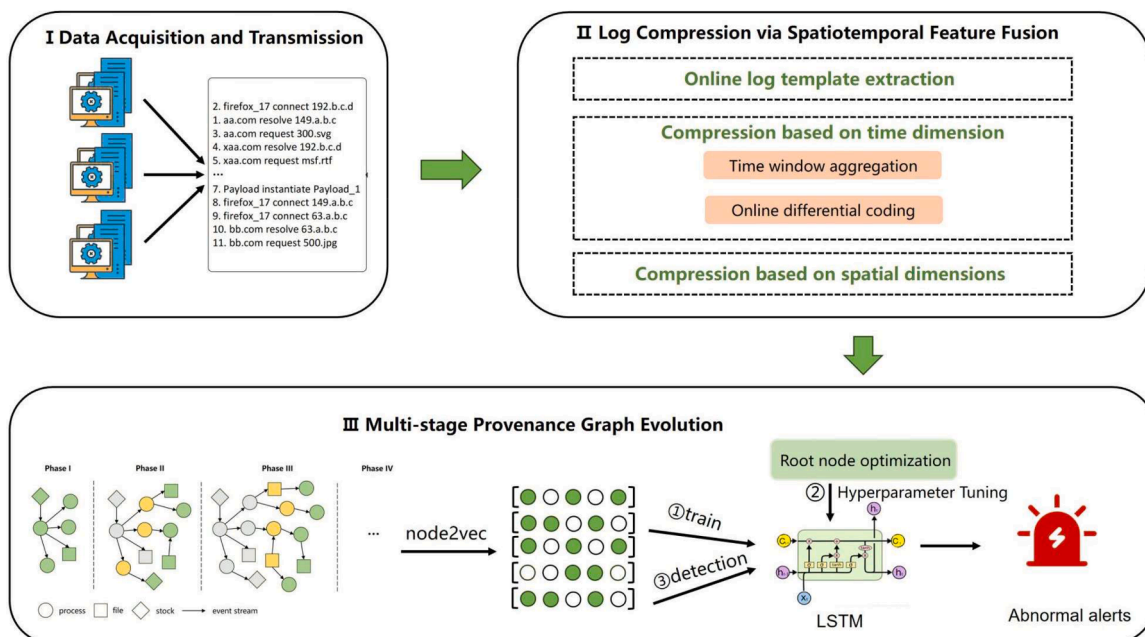


Fig. 2. Architecture diagram of the real-time log compression method based on spatiotemporal features and the evolution of multi-stage provenance graphs.

Node2Vec feature extraction and LSTM temporal modeling to enhance the anomaly identification capability under complex attack scenarios.

The three-stage pipeline is architected to minimize per event latency: 1) the spatiotemporal fusion stage filters noise early to reduce downstream load; 2) the multi-stage graph evolution maintains only active states to ensure $O(V)$ construction time; and 3) the root-node optimized LSTM provides single-pass inference scores. This ensures that the time from log generation to anomaly alert remains well within the actionable window for threat mitigation.

4.1. Data acquisition and transmission

In this paper, SPADE (Gehani and Tariq, 2012) is selected as the core log data collection tool, which not only supports flexible modular design, but also allows users to customize the data filtering logic according to actual needs, so as to achieve accurate recording of specific events and entities. The tool is able to record in detail the interactions between various entities in the system and output the data files in compressed format, which greatly facilitates the subsequent data analysis and in-depth analysis, and the final collected data is mainly divided into two categories:

- (1) Node type: describes entity objects in the system, such as processes, files, sockets, etc.
- (2) Edge types: record interactions between entities, such as inter-process communication, file reads and writes, network connections, etc.

The node data structure covers a variety of types, including common process nodes, file nodes, socket nodes, etc. The process nodes usually contain information such as process number, parent process number, and process creation time, which can demonstrate the hierarchical relationship between processes and execution status. The file nodes contain multiple key attributes for each record, ensuring the integrity and accuracy of the security event analysis and the traceability of the file. The socket nodes focus on the characteristics of network communications, such as source and destination IP addresses, ports, protocol types, etc. It helps monitor abnormal network traffic and provides an important basis for analyzing network interactions. These fields serve as deterministic identifiers for cross-host event stitching, enabling LogSqueeze to maintain behavioral consistency during lateral movement tracking across multiple devices.

The edge data structure serves as a carrier of event records, using a record to represent a specific operation or behavior that occurred in the system, and its core fields usually include the event subject, the event type, and the operation object. In the process of constructing the provenance graph, by finely distinguishing different event types, not only can the interactions between individual nodes be visually depicted, but also nodes in different time periods can be correlated across the time dimension through events that they participate in together, which is crucial for discovering the hidden and intermittent attack behaviors in APTs.

The captured log data will be in dictionary format, for example: {"datum": {"uuid": [121, 105, 84, -91, -36, -24, 99, -90, -80, 88, -115, -32, 104, 79, -82, -37], "sequence": 191, "type": "read", "threadId": 1258, "subject": [-71, -33, 49, 39, -42, 93, -71, -32, -52, 82, -71, 57, -77, 41, 98, -114], "predicateObject": [100, 125, 87, 97, -71, -59, -94, -44, 19, 77, -40, 84, 49, 126, -18, 115], "predicateObjectPath": null, "predicateObject2": null, "predicateObject2Path": null, "timestampNanos": 1618985500764000000, "properties": null}}. This will then be piped to the next module for real-time compression processing.

4.2. Log compression via spatiotemporal feature fusion

1) Online Log Template Extraction

By analyzing the initial data collected by SPADE, it was found that the event data were very long and contained a large number of characters with very low repetition or forensic relevance, in addition to information of significant value for subsequent analysis. Due to invariant structural information in the event records, template extraction was performed for each event. When a new event arrives, by matching the event type with the type in the template Table 1, a successful match will mark the event record as a template number, discarding redundant fields; If it is not possible to match, it will be regarded as a new template for the invariant structure of the event into a new template number and realtime update of the template Table 1.

2) Compression based on time dimension

In this section, two compression techniques, time window aggregation and online differential coding, are utilized in the time dimension to compress logs in the structured-log theme, effectively reducing redundant information in consecutive records and reducing the burden of data storage and transmission.

a) Time window aggregation

When performing time window aggregation operations, the focus is on the processing of duplicate edges to optimize data storage and processing efficiency. If the time window aggregation operation is performed on the events before generating the provenance graph, it not only reduces the amount of events but also improves the efficiency of the anomaly detection algorithm by merging these redundant events.

The causes of duplicate event generation can be categorized into two main cases. One is repeated edges generated by multiple behaviors Fig. 3 (a), r denotes read and t denotes timestamp. This type of repeated edges often reflects similar operations in different behavioral patterns. For example, a user may perform multiple read operations on the same file in different business processes, and the edges generated by these read events are repetitive edges. Since they originate from different behaviors and usually have significantly different timestamps, these repetitive edges can reflect the user's behavioral characteristics and are therefore not aggregated Fig. 3(b).

The other type is repeated edges generated by a single behavior Fig. 3 (c), for example, during data transmission, a large number of repeated transmission events are generated due to continuous data sending or receiving, which in turn form repeated edges, and the merging of these repeated events can more accurately represent user behaviors in terms of frequency, data size, and other characteristics. The difference between t_1 ; t_2 ; t_3 is calculated and then compared with the time window threshold to determine whether aggregation is performed. If the difference is less than the threshold, these repeated events aggregated Fig. 3 (d).

Table 1
Comparison table of log templates.

typology	corresponding numerical value
PROCESS	01
FILE	02
SOCKET	03
EVENT READ	04
EVENT WRITE	05
EVENT FORK	06
EVENT CLONE	07
EVENT ACCEPT	08
EVENT EXEC	09
EVENT CLOSE	10
EVENT CONNECT	11
EVENT CREATE	12
EVENT MMAP	13
EVENT WAIT	14
...	...

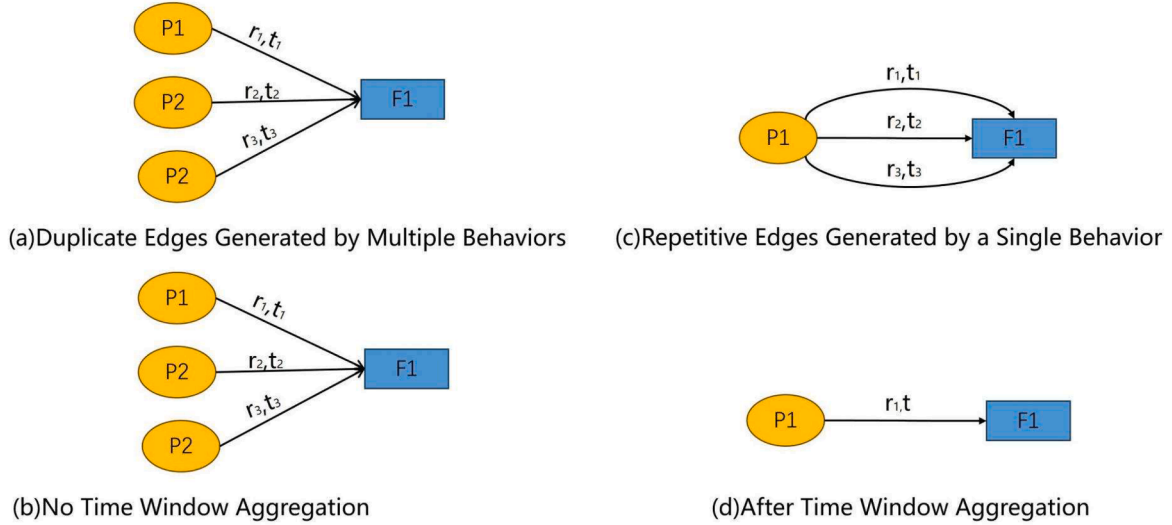


Fig. 3. Example of time window aggregation.

Initially, the initial time window threshold is 1 s. When the occurrence of duplicate events e and e' corresponding to duplicate edges is detected, the difference between their timestamps is compared. If the difference is less than the threshold, these duplicate events are merged. In order to make the time window strategy more adaptable, the time window size is dynamically adjusted after each aggregation operation with the following Eq. (1).

$$\text{threshold}_e = \min(1; 2 * (t'_e - t_e) + 0.001) \quad (1)$$

$\min()$ takes the smallest value in parentheses to ensure that the threshold does not exceed 1, and $2 * (t'_e - t_e)$ multiplies the time difference between repeat event e and e' , $t'_e - t_e$ by 2 as the adjustment basis term for the threshold. Here,

0.001 is a smaller constant term when the two repeat event timestamps are exactly the same, i.e., $t'_e - t_e = 0$, adding 0.001 prevents the threshold from being zero, avoiding a situation where subsequent repeats cannot be processed according.

In addition, the threshold can be further dynamically adjusted in order to adapt to different data traffic. When data traffic surges, the system automatically extends the time window in order to reduce the processing frequency. In low load situations, the time window is shortened in order to improve the real-time data processing. Through the aggregation operation based on the time window, it can not only effectively reduce the number of duplicate edges in the data and lower the data storage burden, but also improve the data processing efficiency to a certain extent.

b) Online differential coding

Online differential coding is a real-time data compression technique used to reduce data redundancy, which focuses on calculating the differences between neighboring data records in real time and storing or transmitting only those differences, not the complete data record. The online differential coding technique is used because monotonic values usually show monotonic growth in continuous log data with a large number of repetitive parts.

In Linux Audit logs, content like 'timestampNanos : 1618985500776000000' often appears, and such timestamps are quite common. Most timestamps in the same log file start with the same digits, and these repeated digits are redundant when stored. Given that such timestamps have the characteristic of monotonically increasing, an online differential encoding approach is adopted for compression. First record the full value of the first monotonic value '1618985500776' as

the initial value (Remove the last 6 digits of the timestamp for high-precision time. Even after removal, the precision still reaches the millisecond level, which meets the requirements of normal log analysis and forensic analysis), calculate the difference between the current timestamp and the initial value, expressed as an offset Δt , $\Delta t = t_{\text{current}} - t_{\text{initial}}$, assuming that the second The difference between the second timestamp and the first is 5 s, then the second timestamp is represented by $t_{\text{initial}} + 5$, and so on, without recording '1618985500781'.

3) Compression Based on Spatial Dimensions

Based on temporal compression, to further spatially compress log data, we employ an online dictionary coding approach to extract high-level features and integrate similar logs. The core of online dictionary coding is to construct and maintain a dictionary that can be updated online to encode and replace frequently occurring strings in logs. These frequently occurring strings include common words, file paths, and so on. Here, we use an in-memory hash table to store dictionary information. When processing log data, the system will look up the current string in the hash table to see if the corresponding encoding already exists. If it exists, the string is directly replaced with the encoding; if a new word is encountered, the system dynamically updates the dictionary mapping, assigns a new encoding to the new word, and adds it to the hash table.

Take the internal office system of an enterprise as an example, it usually uses a unified file storage path to store documents. Suppose that the file server will store all documents in the directory '/data/company/workspace/employees' and that each employee's personal files will be stored in the subdirectory named by the employee ID. When employees perform file operations (such as reading, writing, deleting, etc.), the records of these operations will be recorded in the host log. Due to the large number of employees that frequently operate in the files in their respective directories, the path string '/data/company/workspace/employees' will appear repeatedly in the log, resulting in data redundancy. Using online dictionary encoding, this frequently occurring string can be encoded as '001'. In this way, in the log records, the original lengthy path string can be replaced by '001', which greatly reduces the storage space of log data. This approach only needs to analyze the path entries in the log when storing and maintaining a record of '/data/company/workspace/employees' and '001' corresponding to the rules of the transformation table. After the dictionary encoding process, we just need to store the final processed data including the encoding mapping information (i.e., record the original string corresponding to each

encoding, so that we can decode and restore it when needed), which is convenient for subsequent data processing and storage.

The online dictionary encoding can further achieves spatial compression of data on the basis of temporal compression, while retaining the key information of the data to facilitate subsequent log analysis and processing.

4.3. Multi-stage provenance graph evolution

Multi-stage provenance graph is a graph structure for describing the interaction and causal relationship between entities under different time stages of the system. Utilizing the phase characteristics of the data, incremental storage techniques are used to maintain only the most recent version of each entity when processing the event flow, including new nodes, deleted nodes, and changes in edges, rather than storing the entire historical version of the graph.

1) Dynamic Construction of Multi-stage Provenance Graph

The provenance graph is represented as a directed acyclic graph $G = (V; E)$, where V is the set of nodes, representing various entities in the system, and E is the set of edges, representing interactions between entities, and each newly generated event is taken as the basic unit for constructing the provenance graph during the operation of the system. Let the system generate the event $et = (st; dt; rt; t)$ at the moment t , where st denotes the source entity of the event, dt denotes the target entity, rt denotes the type of the event, and t is the timestamp of the event.

We construct the multi-stage provenance graph as shown in Algorithm 1.

Initial construction phase: The provenance graph is empty in the initial phase, when the first event $e1$ occurs, nodes representing $s1$ and $d1$ are created in the graph, and a directed edge from $s1$ to $d1$ is added, with the attributes of the edge as $r1$ and $t1$. At this point, the provenance graph is initially built to reflect the structure of that initial event.

Algorithm 1

Dynamic construction of multi-stage provenance graph.

Require: Event stream *events*
Ensure: Constructed provenance graph *G*

```

1:  $G \leftarrow \text{empty\_graph}()$  D Initialize empty graph
2: function PROCESS_EVENT_STRGAP(events)
3: for each event  $et = (st; dt; rt; t)$  in events do
4: D Check source entity existence
5: if  $st \notin V(G)$  then
6:  $\text{add\_node}(G; st)$ 
7: end if
8: D Check destination entity existence
9: if  $dt \notin V(G)$  then
10:  $\text{add\_node}(G; dt)$ 
11: end if
12: D Check edge existence
13:  $\text{edge\_exists} \leftarrow \text{false}$ 
14: for each edge  $(u; v; r; \text{timestamp})$  in  $E(G)$  do
15: if  $u = st \wedge v = dt \wedge r = rt$  then
16:  $\text{edge\_exists} \leftarrow \text{true}$ 
17: if need\_update\_edge(edge) then
18:  $\text{update\_edge}(\text{edge}; t)$ 
19: end if
20: break
21: end if
22: end for
23: if not  $\text{edge\_exists}$  then
24:  $\text{add\_edge}(G; st; dt; rt; t)$ 
25: end if
26: D Handle staged provenance graph
27: if need\_staging(dt) then
28:  $\text{create\_stage\_node}(G; dt)$ 
29: end if
30: end for
31: return G
32: end function

```

Regular update phase: As new events continue to be generated, for the event et , different treatments need to be performed depending on whether the source and target entities already exist in the graph:

1. **If both st and dt already exist in the graph:** check if an edge exists from st to dt with attributes rt and t . If it does not exist, the edge is added; if it exists, it is processed accordingly, depending on the specific logic of the event (e.g., some events may update the attributes of the edge). For example, if the event is a multiple write operation to a file, the attributes of the edge need to be updated to record the latest write time and write content.
2. **If one of st or dt does not exist in the graph:** create the non-existing node first, then add the directed edge from st to dt and set the attributes of the edge.

When an entity receives a event, if the node appears with an incoming edge after the outgoing edge, a new node is created, the original node is locked, and the new edge is added only to the unlocked node. For example, if process $P1$ performs a write operation on file $F1$ at moment $t1$, followed by a read operation at moment $t2$, to ensure the time order and loop-free characteristic, if the state of file $F1$ changes after the write operation, the next stage $F1'$ node can be created, and the read operation is associated with $F1'$.

To mitigate the risk of node explosion in high-frequency systems, LogSqueeze adopts an incremental storage strategy. While new stage nodes are logically created to ensure the DAG property and causality, the system only maintains the active state and its latest embedding in memory, reducing complexity to $O(|V|)$. The historical temporal patterns are not "lost" upon discarding previous versions; instead, they are losslessly compressed into the latest node's representation via the recursive embedding mechanism described in the following section.

In this way, the multi-stage provenance graph is constructed step by step, and the above method only stores the latest version of each entity's node and its embedding, instead of the historical version of the whole

provenance graph, and each stage of the graph reflects the state of the system and inter-entity relationships at that moment and before, so that the memory complexity can be reduced from $O(|E|+|V|)$ to $O(|V|)$.

2) Entity Embedding

In this method, vectorized embedding of system entities is performed using the Node2Vec (Grover and Leskovec, 2016). Node2Vec generates a sequence of nodes by performing a biased random walk on the graph, and then learns a low-dimensional vector representation of the nodes using the Skip-Gram (Mikolov et al., 2013) model. For a constructed provenance graph $G = (V; E)$, where V is the set of nodes and E is the set of edges. Starting from each node $v \in V$, a biased random walk is performed. Let the current node be y , the last node visited be z , and the next node to be visited be x . The transfer probability yx from node y to node x is defined as $yx = pq(z; x) \cdot wyx$, where w_{yx} is the weight of the edges between node y and node x (which can be uniformly set to 1 if the edges do not have a set of weights), $pq(z; x)$ is the bias term controlled by parameters p and q as shown in Eq. (2):

$$pq(z, x) = \begin{cases} \frac{1}{p}, & \text{if } d_{zx} = 0 \\ 1, & \text{if } d_{zx} = 1 \\ \frac{1}{q}, & \text{if } d_{zx} = 2 \end{cases} \quad (2)$$

where d_{zx} is the shortest path distance between node z and node x . The parameter p controls the probability of returning to the previous node and q controls the probability of exploring away from the current node. By adjusting the values of p and q , the pattern of random walk can be flexibly controlled to capture different structural features of the graph.

After that, multiple times of biased random walk are performed, starting from each node v . A combination of depth-first search (DFS) and breadth-first search (BFS) is performed to generate a sequence of nodes $\{v_1; v_2; \dots; v_n\}$. Considering these node sequences as sentences in natural language, the vector representation of the nodes is learned using the Skip-Gram model. The goal of the Skip-Gram model is to predict the context node of the center node based on its center node with the objective function of maximizing the log-likelihood function as shown in Eq. (3):

$$L = \sum_{u \in V} \sum_{-k \leq j \leq k, j \neq 0} \log P(v_{u+j} | v_u) \quad (3)$$

Where V is the set of all nodes in the graph, v_u is the vector representation of the center node u , k is the size of the context window, and $P(v_{u+j} | v_u)$ is the probability of predicting the context node $u+j$ based on the center node u , which is usually computed by the Softmax function as shown in Eq. (4):

$$P(v_c | v_u) = \frac{\exp(v_c^T v_u)}{\sum_{v_n \in V} \exp(v_n^T v_u)} \quad (4)$$

where v_c is the vector representation of the context node and v_n is the vector representation of any node in the graph. By training the Skip-Gram model, we finally obtain a low-dimensional vector representation of each entity of the system, and these vectors will be used as input features for subsequent analysis.

3) Anomaly Detection

In the real-time detection phase, the system performs dynamic anomaly analysis by maintaining the latest embedding vector of each entity. When a new event is triggered, the system inputs the latest embedding vector corresponding to the event into the LSTM model pre-trained with benign data for analysis. If the anomaly score output by the

model exceeds the preset threshold, it is determined that anomalous behavior is detected. By introducing the root node optimization strategy, the information of the root node can be incorporated into the update mechanism of the LSTM, thus enhancing the model's ability to capture long-distance dependencies and improving the system's anomaly detection performance in complex attack scenarios. The model will be trained on collected benign system logs, which are analyzed and processed to extract entity information from them and construct the corresponding multi-stage provenance graph. Suppose that the system maintains an internal dictionary D , which maps each node to its latest embedding vector. At each time step t , when a new event $et = (st; dt; rt; t)$ occurs, the vector representation of the relevant node is updated using LSTM. The root node optimization strategy is introduced in the LSTM update process to modify the cell state update formula, and the modified formula is Eq. (5):

$$C_t = f_t \odot C_{t-1} + i_t \odot C_t + c \cdot \left[\frac{1}{n} \sum_{i=0}^n \{r_i\} \right] \quad (5)$$

where $\{r_i\}$ is the set of root nodes associated with element i in the sequence, and c is a learnable model weight to balance the information introduced by the root embedding. For the event et , the hidden state ht of the target node dt is updated using LSTM, which in turn updates its embedding vector $D[dt] = ht$.

In the training phase, the events in the benign training set are sequentially inputted into the LSTM model, and the model automatically learns the values of $weights$ and c , allowing the information from the root node to be effectively incorporated into the cell state update process of the LSTM. After several iterations of training, L dimensional embedding vectors are generated for each entity of the system, and these vectors constitute the set of benign embeddings B representing normal behavior.

In the detection phase, for each new inflow of edge $et = (st; dt; rt; t)$, L dimensional embedding vectors are generated for dt using LSTM and compared with the benign embedding set. The distance between the embedding vector of dt and the nearest neighbor vector in the benign embedding set is calculated, and this distance is used as the anomaly score. If this distance exceeds a predefined threshold, then dt is marked as abnormal.

The source node embedding vector $D[dt]$ and the target node's current embedding vector $D[dt]$ are passed into the LSTM model as inputs to obtain the updated target node embedding vector $D'[dt]$. The formula is shown in Eq. (6):

$$D'[dt] = f(D[s^t]; D[d_t]) \quad (6)$$

where f is the LSTM function. We then calculate the cosine distance d_{cos} between the updated embedding vector $D'[dt]$ and the nearest neighbor embedding vector in the benign embedding set B . The cosine distance d_{cos} formula is shown in Eq. (7):

$$d_{cos} = 1 - \frac{D'[dt] \cdot B_{nn}}{\|D'[dt]\| \cdot \|B_{nn}\|} \quad (7)$$

Where B_{nn} denotes the nearest embedding vector to $D'[dt]$ in the benign embedding set B , and $\cdot$ denotes the dot product of the vectors, and the target node dt is labeled as an anomaly if the cosine distance d_{cos} exceeds the predefined threshold as shown in Eq. (8):

$$\text{if } d_{cos} > \text{then Mark}(d_t; \text{Abnormal}) \quad (8)$$

With the above approach, the multi-stage provenance graph evolution technique is able to effectively record and analyze causal relationships in a system, supporting realtime safety monitoring and fault troubleshooting, while significantly reducing storage costs and improving detection accuracy.

4) Process Multi-stage Provenance Graph Evolution

In order to better understand the process of multi-stage provenance graph evolution, the following demonstrates the entire process through an APT attack example, which is shown in Fig. 4.

The attack process is as follows: the attacker delivers malware disguised as PDF documents via phishing emails, utilizes the vulnerability of Adobe Reader to trigger the initial process P_1 , P_1 loads the malicious $DLL(exploit.dll)$ and creates the sub-process $P2(cmd.exe)$, the process P_2 downloads the encrypted load through C2 server S_1 (203.0:113:5),

P_2 decrypts it and starts $P3(rundll32.exe)$, which exploits a kernel vulnerability to elevate privileges to SYSTEM, and P_3 creates the hidden process $P4(svchost.exe)$, which disguises itself as a system service. P_4 moves laterally to the intranet host 10:0:0:2, and exploits the vulnerability to create the remote process $P5(winlogon.exe)$, which is used to create the remote process $P5(winlogon.exe)$ and steals the target host's F_1 file (C:\\SAM) and sensitive document F_3 (C:\\project.xlsx), generating a copy of $F_2(sambackup.bin)$. P_5 compresses the data and uploads it via S_1 , while P_4 deletes the security log F_4 and forges the empty file F_5 . Throughout the process, the user downloads the update using Chrome normally, generating benign processes P_6 and P_7 .

Yellow nodes in the graph indicate benign nodes and blue nodes indicate malicious nodes. Phase I-VI show how the multi-phase provenance graph evolves to ensure a smaller memory footprint by maintaining only the current phase state of system entities. Taking phase V as an example, the sensitive file F_3 of the target host stolen by process P_5 is denoted as $x_{10}': x_{10} + x_{7''}$, where x_{10} is the feature vector of file F_3 and $x_{7''}$ is the recursive embedding of the events involved before process P_5 . This recursive operation effectively reduces the computational and memory cost of the system, thus achieving log compression while preserving accuracy.

As illustrated in Phase V (Fig. 4), the embedding x_{10}' for file F_3 is calculated as $x_{10} + x_{7''}$. Here, $x_{7''}$ acts as a temporal summary of all prior events associated with process P_5 . This recursive operation ensures that even as older nodes are pruned to save space, the long-range behavioral dependencies essential for anomaly detection remain embedded within the current feature vector, effectively balancing storage efficiency with forensic depth.

5. Evaluation

5.1. Dataset

In this paper, we use the DARPA TC E3 dataset (Darap3, 2023), the Darpa TC E4 dataset, and the real-world deployed cyber range datasets in our laboratory environments. As shown in Table 2.

The DARPA TC E3 dataset was generated during the third Red Team

Table 2
Experimental dataset settings.

Dataset	Source System	Data Source	Attack Description
D-1	Ubuntu12.04	Darpa E3	Firefox browser vulnerability backdoor implantation
D-2	Ubuntu14.04	Darpa E3	Malicious executable file attack
D-3	FreeBSD	Darpa E3	Phishing email attack
D-4	Windows 10	Darpa E3	Sending macro virus to steal victim host files
F-1	Ubuntu14.04	Darpa E4	Information leakage & malicious file download and execution
F-2	Ubuntu14.04	Darpa E4	Firefox memory attack
T-1	Ubuntu16.04	Lab	Webshell attack
T-2	Ubuntu16.04	Lab	Remote Access Trojan (RAT) attack
T-3	Ubuntu16.04	Lab	Fileless attack

and Blue Team adversarial encounter in April 2018, which consisted of system data collected from individual hosts, with both benign activity on each host as well as Red Team's attacks. DARPA TC E3 dataset contains the Theia dataset (D-1), Trace dataset (D-2), Cadets dataset (D-3) and FiveDirections dataset (D-4). D-1 focuses on backdoor attacks implanted through vulnerabilities in the Firefox browser; D-2 targets attacks inducing the execution of malicious executables; D-3 captures attacks sending phishing emails through mail servers; and D-4 documents APT attacks that utilize documents containing malicious macros for phishing.

The DARPA TC E4 dataset was derived from data from the fourth DARPA engagement, and the experiments are conducted on two hosts running Ubuntu 14.04 systems for data collection and APT attack simulation experiments. During the Red Team's attack on the hosts, normal program operations were running on the hosts at the same time, including browsing the web, downloading and running binary files, sending and receiving emails, and editing documents. More than 99% of the events in the dataset are non-attack related events, and this dataset is further subdivided into two sub-datasets, F-1 and F-2. F-1 documented a multi-stage composite attack including credential theft for information gathering, malware implantation and execution disguised as CCleaner, and elevation of privilege and scp transfer via hc malicious files; F-2 focused on a memory attack exploiting a Firefox vulnerability by tampering with system files, creating a malicious sshd process to achieve data outflow, and ultimately executing the implanted F-2 focuses on memory attacks that exploit Firefox vulnerabilities by tampering with system files, creating malicious sshd processes to export data, and eventually executing the implanted malicious payload.

In order to simulate APT attacks, the experimental dataset acquisition process follows the typical cyber kill chain model to design the attack path, which consists of approximately seven phases, namely,

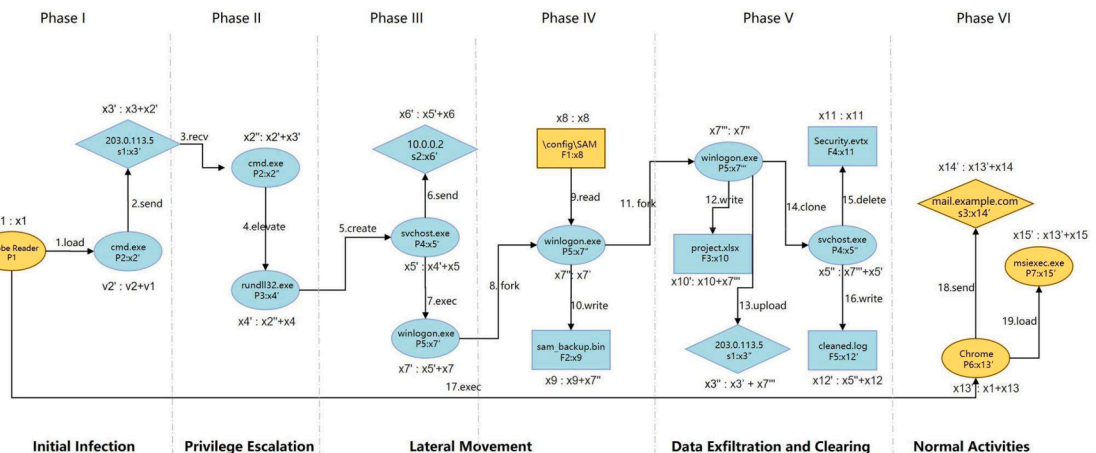


Fig. 4. Example of APT penetration attack.

reconnaissance phase, weaponization phase, delivery phase, exploitation phase, installation phase, command and control phase, and action against the target phase (Hutchins et al., 2011). The laboratory data were collected in a controlled environment using the SPADE acquisition tool, i.e., after deploying SPADE on the Ubuntu 16.04 operating system, three datasets, T-1, T-2, and T-3, were generated by simulating three APT attack scenarios. T-1 is an attack that utilizes Web file upload vulnerability to implant Webshell and achieve privilege elevation by abusing privilege flaws in timed task scripts; T-2 is an attack that uses social engineering to induce the execution of a Trojan horse to obtain initial privileges and then utilizes the timed task mechanism to tamper with the system script to elevate privileges; and T-3 is a living-off-the-land attack targeting the buffer overflow vulnerability of special service port 29,273 to implement a combination of timed task tampering and trace removal to complete privilege escalation. All three of them achieve privilege elevation through scheduled tasks, and implement subsequent attacks such as credential theft, data outflow and persistence control after privilege elevation.

By analyzing the above dataset, the characteristics of the dataset are obtained as shown in Table 3.

5.2. Parameter experiment and analysis

In this section, the following six aspects will be analyzed: the selection of RNN model, the number of model layers, the amount of training data, the degree of optimization of the root node, the size of the feature vector, and the selection of the time window threshold. In order to test the various model parameters more comprehensively, the test will be conducted using the T-1 collected in the laboratory, which contains the complete seven steps of the network kill chain model, covers a more comprehensive attack path, and has higher data quality and integrity, which can effectively support the validation process of the methods in this section. Each figure in the experiment demonstrates the Receiver Operating Characteristic (ROC) curve, which is a graphical tool for evaluating the performance of a classification model by comparing the true positive and false positive rates of the system at different classification thresholds. The Area Under the Curve (AUC) is a measure of the overall performance of the model, with a value between 0 and 1. The AUC is a graphical tool for evaluating the performance of a classification model by comparing the true positive and false positive rates of the system at different classification thresholds. In the following, experiments will be conducted on each of the six components of our paper.

1) RNN Model Selection

In the research of real-time threat detection for network security, the selection of appropriate RNN models is extremely critical to accurately capture the sequence patterns and causal relationships of system behaviors. Theoretically, sequence-based neural networks are a good fit for this approach because they can model complex attack behaviors with limited underlying sequence information. Among the RNN variants, GRU (Cho et al., 2014) and LSTM are able to solve the problem of memory loss due to gradient disappearance in traditional RNNs by

introducing a gating mechanism, and they perform well in handling long sequence data: the GRU model is simplified, and the gating mechanism allows it to selectively update the reset information, which overcomes the limitations of traditional RNNs, while the LSTM is more advantageous in coping with long-term dependency relationships and complex sequence patterns.

While modern graph-based models such as GNNs, Graph-SAGE, and TGNs have shown promise in static or offline graph analysis, LogSqueeze prioritizes low-latency streaming processing to meet the average breakout time of 48 min in modern APTs. Newer models often involve complex multi-hop neighborhood aggregation or continuous-time convolutions, which introduce significant computational overhead and latency during real-time ingestion. By utilizing Node2Vec for structural feature extraction and recursive embedding to compress historical causal logic, we effectively transform the dynamic graph evolution into a sequential dependency problem. This allows LSTM to achieve a processing throughput of 52 K entries/s, ensuring the system stays well ahead of data generation rates while maintaining high accuracy.

In order to visually compare the performance of LSTM and GRU, experiments were conducted on the T-1 dataset constructed independently by the lab. While keeping the other model parameters consistent, the comparison results (e.g., Fig. 5) clearly show that, even though the length of some of the attack sequences in the T-1 dataset is not particularly long, LSTM still demonstrates a far superior performance to GRU, with an AUC value of 0.76, while the AUC value of GRU is only 0.58. In real-world cyber threat detection scenarios, where attack behaviors often have complex time-series features and long-term dependencies, this advantage of LSTM will be even more obvious. Therefore, **LSTM model** is chosen for this method.

2) LSTM Layers

Previous research (Carlini et al., 2022) emphasized the effect of model size on performance, and in this section, experiments were conducted on LSTM models with different numbers of layers. By systematically tuning the number of LSTM layers (1, 5, 10, and 20), significant performance trends were observed, as shown in the experimental results Fig. 6. The experimental results show that increasing the number of layers improves the classification performance, but also increases the computational resources, when the number of layers is increased from 1 to 5, the AUC value increases dramatically from 0.40 to 0.58, which suggests that it is difficult for the shallow network to model the complex attack paths in the dataset effectively. As the number of layers further increases to 10, the AUC reaches a peak value of 0.75, at which point the model is able to capture the long-distance dependencies in the attack behaviors through a deeper hierarchical structure, however, when the number of layers increases to 20, the AUC slightly decreases to 0.69, which may be due to the overfitting that causes the model to be over-sensitive to the noise in the training data. Considering the performance gain and computational cost, this method selects **10-layer LSTM** as the baseline configuration for subsequent experiments.

3) Root Node Optimization Strategy

The difficulty of LSTM in modeling long-term dependencies is alleviated by directly embedding root node information in the local context of each entity. Root node information represents the root factors of system behavior and can provide key clues for understanding and analyzing attack paths. This section tests the model performance with and without root node optimization, respectively. From the experimental results shown in Fig. 7, it can be seen that the model classification performance is significantly improved after incorporating root node optimization. Specifically, the AUC value increases from 0.60 to 0.72, and this performance improvement is more obvious in the FPR region. This implies that incorporating the **root node optimization strategy** can help the model identify attack entities more accurately while

Table 3
Characteristics of datasets.

Dataset Label	Node Count	Edge Count
D-1	394,388	7212,228
D-2	457,553	8311,863
D-3	498,547	5123,658
D-4	732,487	11,296,984
F-1	351,867	3693,252
F-2	428,300	4524,260
T-1	132,457	1068,953
T-2	162,587	1141,039
T-3	86,785	958,924

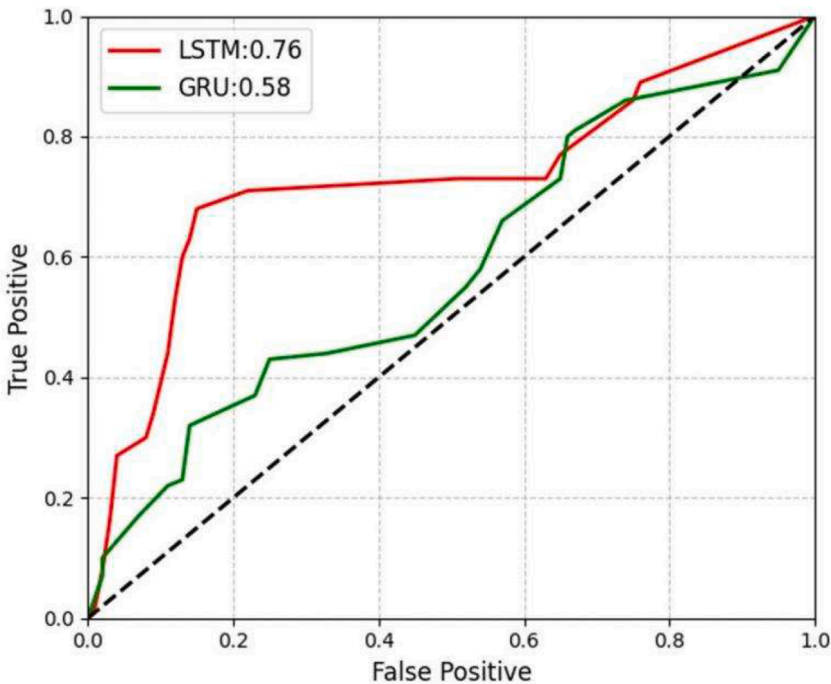


Fig. 5. Comparison experiment of RNN models.

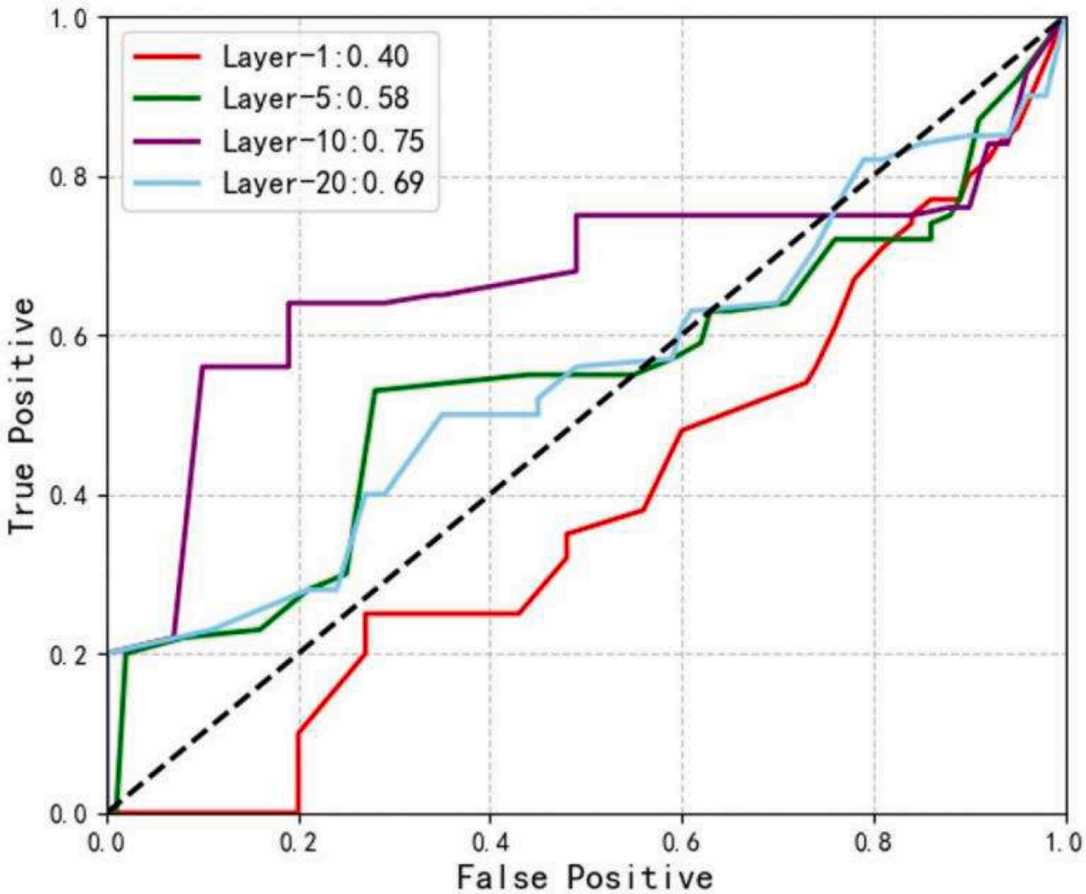


Fig. 6. Comparison experiment of different numbers of layers in LSTM.

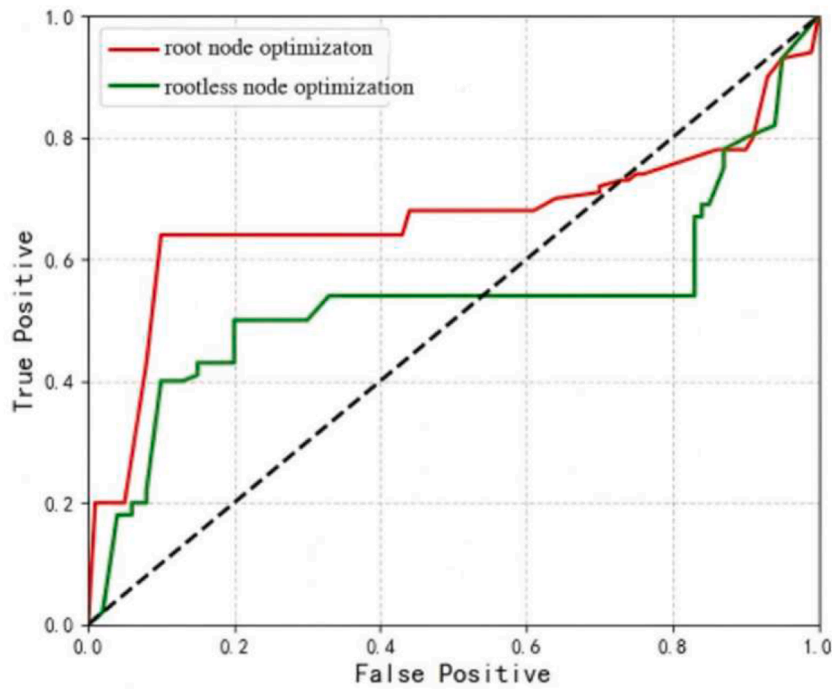


Fig. 7. Comparison experiment of the optimization strategy with and without root nodes.

ensuring low false positives.

4) Feature Vector Dimension

In LSTM models, feature vectors act as numerical representations of each system entity and are extremely critical parameters. While richer feature vectors may better capture the underlying data distribution, the inclusion of additional information may also introduce noise that interferes with model judgment. In this section, feature vectors of different

dimensions are tested, set to 32, 64, 128 and 256 dimensions, respectively. As can be seen from the experimental results shown in Fig. 8, feature vectors of different dimensions have a significant impact on the model's classification performance, with the **128-dimensional vector** obtaining the

Most favorable ROC curve (0.68 AUC), and thus this size is used in the rest of the experiments in this method.

5) Volume of Training Data

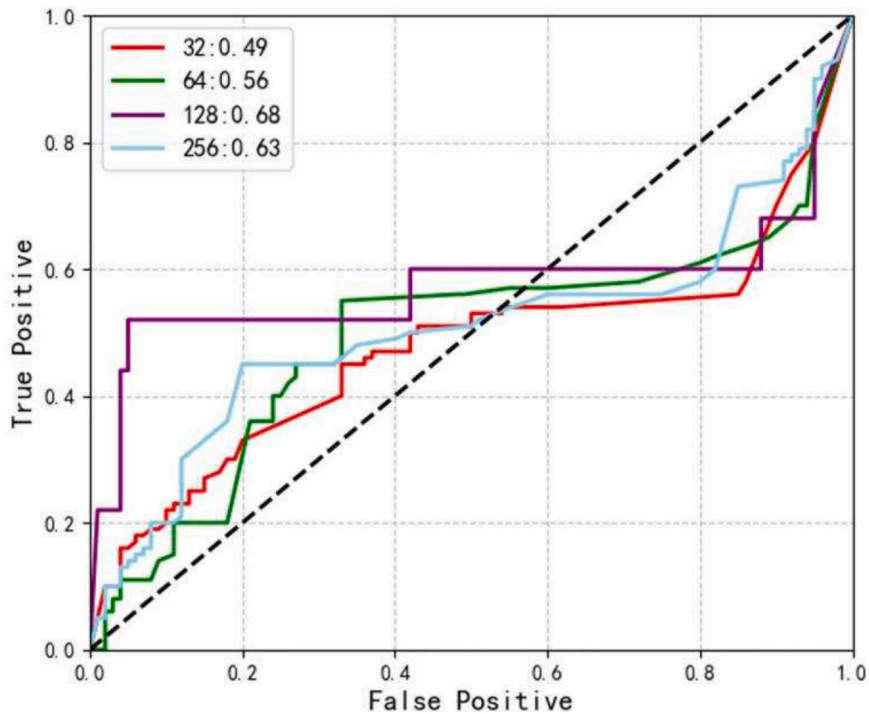


Fig. 8. Comparison experiment of different feature vector dimensions.

In the detection process, the size of the training dataset plays a very important role in the performance of this method. In order to achieve a higher model detection rate for efficient analysis of real-time data, it is especially critical to determine an appropriate training dataset size. To determine the appropriate training dataset size, this experiment is tested using the larger D-1 dataset. Starting with a training dataset of 100 K edges, it is gradually increased to 5 M edges, where 5 M edges roughly correspond to one day's log records on the host system, and the experimental results are shown in Fig. 9.

The overall upward trend in classification performance with increasing training data suggests that more training data helps the model learn more accurate patterns and improves its ability to recognize abnormal behavior. Although the AUC of 1 M events is higher than that of 5 M events, 5 M events are roughly equivalent to the activity volume of the host system for a whole day, and can better represent the various types of behavioral patterns in the actual operation of the system. In contrast, 1 M events have a relatively small amount of data and may not be able to comprehensively cover all kinds of situations in system operation. After comprehensive consideration, this method chooses to use select 5 M events to train the LSTM model (0.92 AUC) in the subsequent experiments. When facing different datasets or actual application scenarios, 5M events can provide sufficiently rich information for the model, and it is more likely to show good generalization ability to accurately detect abnormal behaviors without causing omissions or false alarms due to insufficient data coverage.

6) Time Window Thresholds

To ensure the efficiency of log compression, this section tests different time window thresholds. The Table 4 and Fig. 10 show the compression ratios and the F1 score values of the LSTM models in this chapter when the initial time window size is set to 0.01 s, 0.1 s, 1 s, 10 s, and 100 s on the T-1 dataset, respectively. Where F1 score is a core metric used in the field of machine learning to measure the comprehensive performance of a classification model, the higher the F1 value, the more balanced the model's ability to classify positive and negative samples in the anomaly detection task, which is given by the Equation9:

Table 4

The influence of different time window sizes on the compression ratio.

Time Window (s)	0.01	0.1	1	10	100
Dynamic Time Window	$6.8 \times$	$7.2 \times$	$7.5 \times$	$7.7 \times$	$8.1 \times$
Fixed Time Window	$6.8 \times$	$7.3 \times$	$7.7 \times$	$8.1 \times$	$8.5 \times$

$$F1 = 2 \times (\text{Precision} \times \text{Recall}) / (\text{Precision} + \text{Recall}) \quad (9)$$

In addition to this, the effect of the size of the fixed and dynamic time windows on the threshold selection was measured. It can be observed that the compression efficiency and generaliz ability of the dynamic group is less affected by the threshold selection as compared to the fixed group. Moreover, setting the time window size to 1 s ensures better efficiency and generalizability. While Fig. 10 shows that the time window size indeed influences the F1 score and compression ratio, the dynamic adjustment strategy significantly mitigates this sensitivity compared to fixed-window approaches. As observed in Table 4, the performance variance in the dynamic group is considerably lower, with the compression ratio remaining relatively stable ($6.8 \times$ to $8.1 \times$) even as the initial threshold scales by four orders of magnitude. This robustness suggests that by linking the threshold to real-time event intervals, LogSqueeze automatically adapts to traffic fluctuations, thereby minimizing the reliance on manual parameter tuning and ensuring stable performance in diverse enterprise environments.

5.3. Performance of data compaction

Since CPR, F-DPR and GS all compress the data based on pruning the duplicate edges, these methods are more similar to the work in this paper and are therefore chosen as comparison methods.

Table 5 shows the compression ratio of LogSqueeze with CPR, GS, and F-DPR, and the compression ratio is defined in Eq. (10):

$$\text{Compressionratio} = \frac{\text{originaldatasize}}{\text{compresseddatasize}} \quad (10)$$

In compression of host logs, previous work lacks exploration of duplicate edge scenarios. CPR removes only duplicate edges with fully

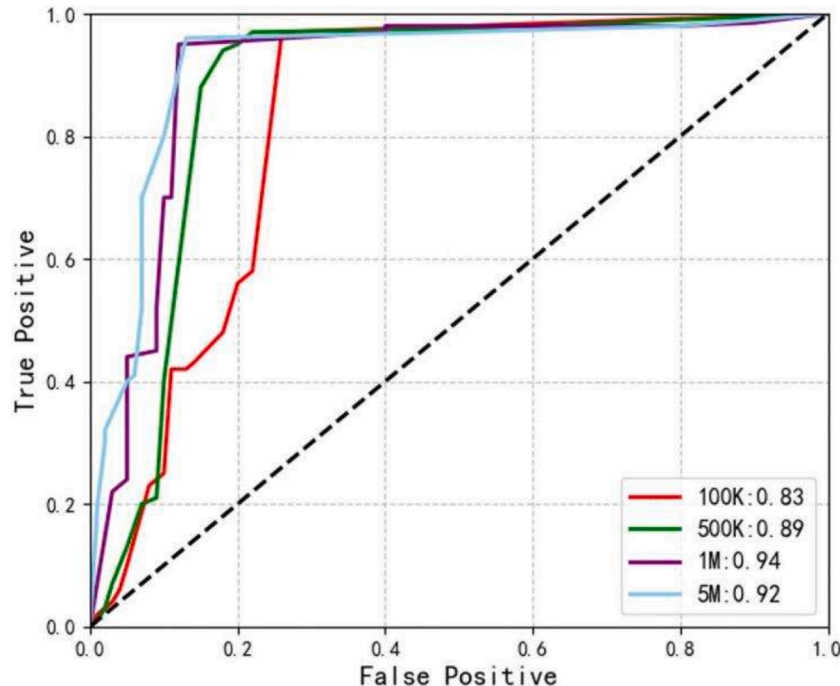


Fig. 9. Comparison experiment of different training data volumes.

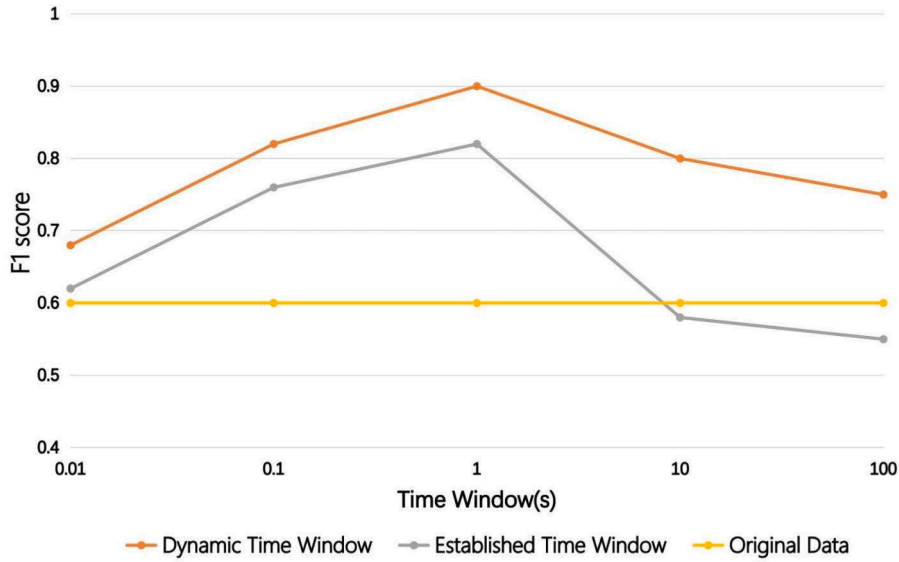


Fig. 10. The influence of different time thresholds on the F1 score in the LSTM model.

Table 5

Log compression effectiveness evaluation.

Dataset	CPR	GS	F-DPR	LogSqueeze
D-1	3.6 ×	4.8 ×	2.6 ×	5.6 ×
D-2	4.8 ×	4.7 ×	4.2 ×	4.9 ×
D-3	4.9 ×	7.3 ×	5.7 ×	6.8 ×
D-4	6.2 ×	9.2 ×	6.5 ×	9.2 ×
F-1	5.2 ×	5.9 ×	4.8 ×	7.7 ×
F-2	4.4 ×	4.5 ×	3.5 ×	6.3 ×
T-1	7.9 ×	10.2 ×	9.4 ×	12.6 ×
T-2	6.6 ×	9.6 ×	7.6 ×	11.8 ×
T-3	6.2 ×	8.2 ×	8.8 ×	8.9 ×

overlapping causal dependencies, GS and F-DPR remove duplicate edges that do not contribute to subsequent dependencies on top of removing duplicate edges with fully overlapping causal dependencies, thus achieving higher compression efficiency while maintaining data integrity. In contrast, LogSqueeze is able to achieve higher compression ratios (4.9–12.6×) and significantly outperform methods such as CPR, GS, and

F-DPR by employing spatiotemporal analysis.

When analyzing the efficiency of log compression, it is found that the compression efficiency of the laboratory datasets T-1, T-2, and T-3 is significantly higher than that of the Darpa dataset due to the difference in the granularity of data collection. It may be due to the fact that in the laboratory environment, a large number of identical behaviors are performed in a short period of time, such as a large number of read and write operations, while the Darpa dataset lasts longer, involves more nodes, and the interactions are more complex and closer to the actual scenarios.

5.4. Real-time analysis

the data transfer rate reaching about 430 entries per second, while it is relatively smooth when the host is idle. Meanwhile, the data consumption rate of the system performs well, processing about 316 K pieces of data in 6 s, and the average processing rate reaches 52 K pieces per second.

We evaluate the real-time processing capability of LogSqueeze This result indicates that the system's processing power is by comparing the

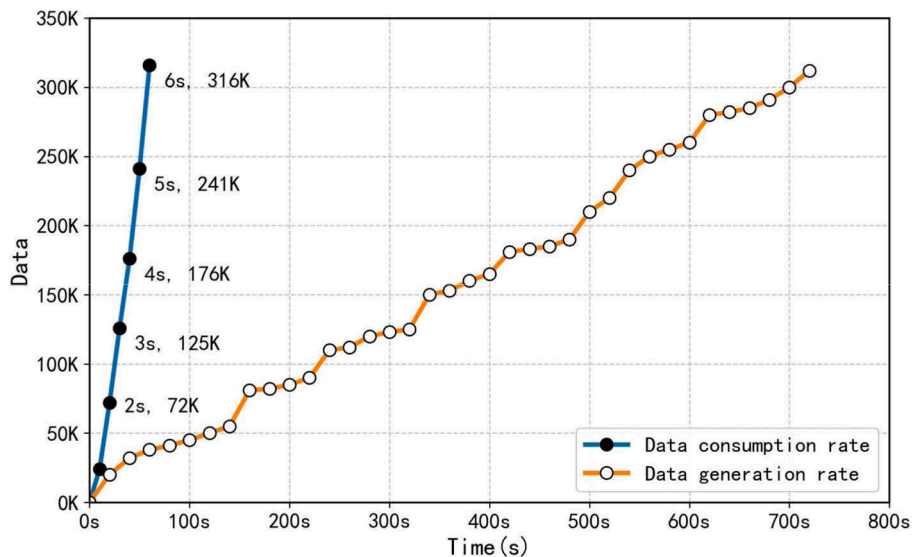


Fig. 11. Comparison experiment of real-time data generation and consumption rates.

rate of data consumption and the rate of generation. In the experiment, we performed normal operation on a Linux host for 720 s, simulating scenarios similar to the nine datasets mentioned in Section V A. The results of the experiment are shown in Fig. 11.

During the experiment, the system generated and transmitted about 310 K pieces of data (including entities and events) in real time. During the data transfer process, the data generation curve is steeper when the host is busy, with significantly higher than the data generation rate, and can effectively cope with the fluctuation of data traffic.

Further analysis shows that the stream event stream processing platform can still maintain stable performance under high load. By dynamically adjusting the time window threshold and other mechanisms, the system successfully controls the processing delay at the millisecond level, ensuring real-time performance. This efficient processing capability enables LogSqueeze to respond and process large amounts of log data in a timely manner, providing strong support for real-time security monitoring.

5.5. Generic analysis

In order to further verify that the compressed data does not have adverse effects on anomaly detection, the experiments select the F-1 and F-2 datasets as the research objects, which contain a variety of attack scenarios and can provide a rich testing environment for the experiment. The original datasets underwent initial analysis. All anomalous events within them were accurately identified using the officially provided Ground Truth. Subsequently, experiments were conducted with other existing works in the field of anomaly-based intrusion detection. These experiments utilized data compressed via the methods detailed in this paper, as well as data compressed using the CPR method.

In order to evaluate the impact of compressed data on anomaly detection results, Threatrace (Wang et al., 2022), Kairos (Cheng et al., 2024), and the anomaly detection method in our paper are selected as representative methods for comparative experiments. Beyond these baselines, our discussion also situates LogSqueeze within the context of recent breakthroughs in audit synchronicity (Gandhi et al., 2023), graph-native embedding (PROGRAPHER (Yang et al., 2023)), and lightweight behavior modeling (ProvGOutLiner (Wang et al., 2025)). These works represent the current state-of-the-art in balancing forensic coverage with operational efficiency. Threatrace uses graph neural network (GNN) and multi-modeling framework to construct a classification model for each type of node in the graph and achieves node-level misclassification of labels by modeling the anomaly detection. While Kairos uses TGN and MLP as codecs to analyze the temporal evolution and contextual structure of the provenance graph, to quantify event anomalies in real-time and to detect and reconstruct attack paths in combination with community discovery techniques.

The experimental results are shown in Fig. 12, where All denotes the detection using the original dataset, Our denotes the detection using the dataset compressed by the method in our paper, and CPR denotes the detection using the data compressed by the CPR. LogSqueeze demonstrates excellent detection performance on the F-1Dataset and F-2Dataset. On F-1Dataset, its F1 score is 0.06 higher than the All and 0.02 higher than the CPR. On F-2Dataset, the F1 score is 0.04 higher than the

All and 0.03 higher than the CPR. Meanwhile, LogSqueeze achieves an average precision of 0.66, an average recall of 0.94, an average accuracy of 0.93, and an average F1 score of 0.77 on the two datasets. Its overall performance outperforms both the All and the CPR, highlighting the significant superiority of detection performance on compressed data.

The main reason is that the compression method proposed in our paper more accurately reflects the event patterns in different scenarios: for example, in the file modification operation, multiple consecutive or alternating read/write events are efficiently merged into key logs, which not only reduces the redundant interference, but also strengthens the typicality of the behavioral patterns, thus significantly improving the model's ability to identify abnormal behaviors. For the Kairos experiment (shown in Fig. 13), on the F-1Dataset, the recall reaches 0.98, which is 0.03 higher than 0.95 of the ALL; the F1 score is 0.80, 0.05 higher than 0.75 of the ALL; and the accuracy is 0.90, only 0.01 higher than that of the ALL. On the F-2Dataset, the recall is 0.90 and the accuracy is 0.92, which are consistent with those of the ALL.

Our choice is further validated by comparisons with GNN-based Threatrace and TGN-based Kairos. As shown in Figs. 12 and 13, although these nearest models utilize more complex graph-native architectures, LogSqueeze achieves comparable or even superior F1 scores and recall on compressed data. This suggests that for real-time APT detection, the combination of high-quality spatiotemporal compression and efficient sequential modeling provides a more optimal balance between resource consumption and detection performance than computationally expensive graph-neural architectures.

Although the accuracy of compressed data for detection is the same as that of using the original data, the compressed graph scale is smaller in practical applications, and the number of logs to be reviewed by security personnel is significantly reduced, which in turn improves the efficiency of anomaly analysis. This shows that LogSqueeze ensures the core detection metrics while significantly optimizing the user experience in real world deployment by intelligent compression.

For the experiments using the detection method proposed in our paper (shown in Fig. 14), on F-1Dataset, the F1 score of LogSqueeze (Our) reaches 0.69, which is 0.02 higher than 0.67 of the All and 0.01 higher than 0.68 of the CPR. On F-2Dataset, the F1 score is 0.63, which is 0.02 higher than 0.61 of the All and 0.02 higher than 0.61 of the CPR. Additionally, the accuracy of LogSqueeze (Our) on F-1Dataset is 0.84, which is 0.04 higher than 0.80 of the All and 0.01 higher than 0.83 of the CPR. On F-2Dataset, the accuracy of the LogSqueeze (Our) reaches 0.92, which is improved by 0.03 and 0.08 compared to 0.89 of the All and 0.84 of the CPR, respectively.

The results highlight the advantages of compressed data in our detection method, not only in the detection accuracy has not been reduced due to data compression, but on the contrary, by optimizing the data structure, it makes the model able to identify the anomalous patterns more efficiently. Meanwhile, LogSqueeze significantly reduces the amount of data to be processed on the basis of ensuring detection accuracy, reduces computational cost and storage requirements, and improves the operational efficiency and scalability of the detection system.

The above three sets of experiments show that LogSqueeze has no significant impact on the anomaly detection logic in different detection frameworks and application scenarios, and is able to significantly

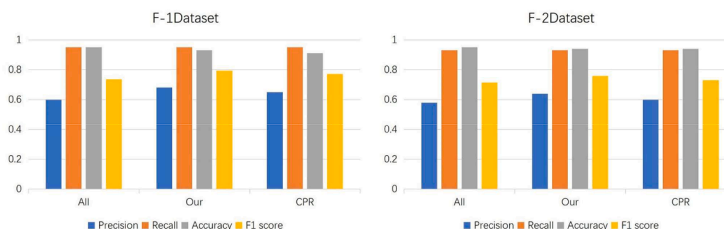


Fig. 12. Results of anomaly detection in Threatrace by different compression methods.

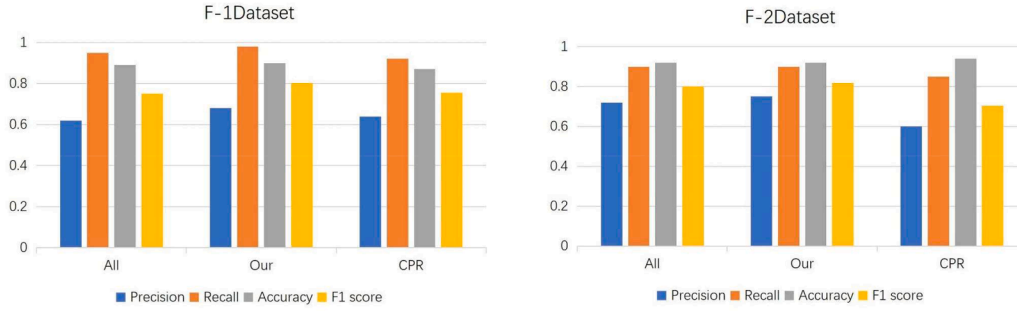


Fig. 13. Results of anomaly detection in Kairos by different compression methods.

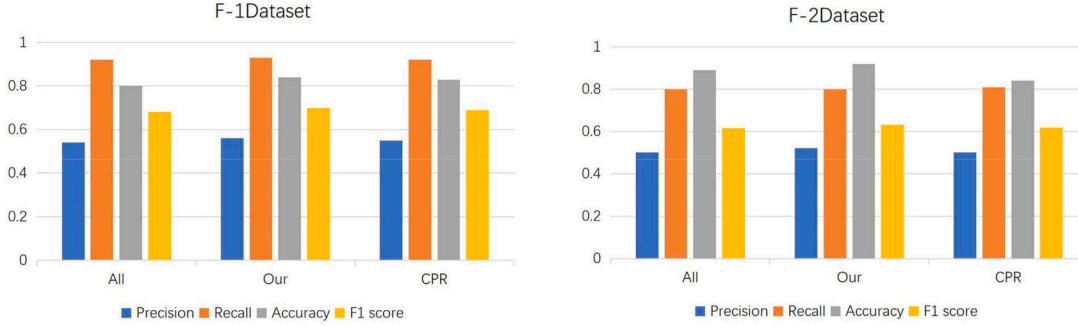


Fig. 14. Detection results of different compression methods on the detection methods in our paper.

optimize the practical application effect of the anomaly detection system without sacrificing the detection performance. Compared to the latest SOTA such as PROGRAPHER (Yang et al., 2023) and ProvGOutLiner (Wang et al., 2025), which respectively emphasize high-fidelity structural embedding and process-level lightweight features, LogSqueeze provides a unique architectural advantage. By implementing aggressive spatiotemporal reduction upstream, our system addresses the data explosion problem that remains a scaling bottleneck even for these modern detection frameworks, ensuring that high-performance detection remains actionable in massive-scale enterprise environments. LogSqueeze uses a combination of multiple compression strategies to effectively reduce noise interference, provide better quality input features for graph neural networks, and ultimately achieve a double breakthrough in detection performance and storage efficiency, which fully reflects the robustness of the method in multi-scenario applications.

6. Discussion

In this section, we discuss the limitations and improvements of our work.

6.1. Taint explosion

The compression effectiveness of LogSqueeze is inherently linked to the propagation rate of tainted data. Rapid taint explosion—where a single suspicious event leads to the tracking of numerous subsequent benign interactions—can significantly degrade the compression ratio.

To provide a practical solution, we propose a Multi-level Trust-Propagation (MLTP) mechanism. Instead of a static whitelist, LogSqueeze assigns dynamic confidence scores to system entities. For instance, high-frequency but low-risk background services (e.g., legitimate system updates or telemetry from verified IP ranges) are assigned a high trust score. When these entities participate in communications, the system applies Selective Depth Tracking, which terminates the taint propagation chain for these verified services after a predefined threshold. This ensures that routine system activities do not trigger a

massive creation of redundant stage-nodes, thereby maintaining a high compression ratio (as evidenced by our $12.6 \times$ peak in T-1) even in complex network environments.

6.2. Multi-host distributed correlation

While the current implementation of LogSqueeze primarily focuses on single-host log compression, its architecture is designed to support the detection of lateral movement through Distributed Event Stitching.

Currently, LogSqueeze employs a deterministic Global Network Tuple Correlation strategy, which is fully implemented and serves as the backbone for our multi-host analysis. By utilizing socket event attributes (Source IP, Destination IP, Port, Protocol, and Timestamp) as join keys, LogSqueeze can correlate outbound edges from a compromised host with inbound edges on a target host. Specifically, the multi-stage provenance graph is capable of treating cross-host socket nodes as bridge entities. When an attack initiates lateral movement, therecursive embedding from the source host is encapsulated into the socket node's feature vector. Upon reaching the destination host, this vector serves as a pre-tainted context, allowing the detection model to maintain causal continuity across the network boundary. This architectural integration transforms the system from a localized monitor into a collaborative defense fabric, effectively addressing the temporary nature of time-based heuristics. While decentralized structures like Bloom filters are mentioned as potential candidates to further reduce synchronization overhead in future global-scale distributed clusters, the existing deterministic matching provides a robust and verified solution for the environments tested in this study.

6.3. Parameter sensitivity and automation

The current adaptive threshold heuristic is a complete and robust implementation that successfully stabilizes system performance under varying traffic loads. Despite the advantages of our adaptive strategy, the initial threshold setting still acts as a prior that influences the system's convergence. In high-frequency environments, a suboptimal

initial value might lead to a temporary lag in compression efficiency. To further reduce this dependence, future iterations of LogSqueeze could incorporate online learning. The suggestion of online learning is intended as an evolutionary enhancement for highly non-stationary environments where attack patterns shift over years, rather than a necessary component for the system's current real-time operational capacity.

6.4. Model dependency and data requirements

The effectiveness of LogSqueeze's anomaly detection module currently relies on the availability of a clean training history. We acknowledge that in some volatile environments, obtaining sufficient benign data for LSTM training can be challenging. However, LogSqueeze is designed as a modular architecture where the compression engine and detection engine are decoupled.

To mitigate training costs and data sensitivity, several strategies can be employed. 1) Data-Efficient Learning: Our spatiotemporal compression reduces the training corpus volume by up to $12.6\times$, which inherently lowers the computational cost and time required for LSTM convergence. 2) Transfer Learning: For environments lacking a long history of benign logs, pre-trained models from similar host profiles can be utilized as a baseline, followed by minor fine-tuning on local data. 3) Unsupervised Adaptation: Future work will explore the integration of unsupervised or semi-supervised models that can learn to identify anomalies without a strictly clean start, ensuring the system's robustness in cold-start or highly dynamic deployment scenarios.

6.5. Real-world scalability and multi-system environments

While our evaluation leverages high-fidelity datasets such as DARPA TC and laboratory cyber ranges, we acknowledge that these controlled environments do not fully capture the complexity of a production network with millions of heterogeneous devices. The primary rationale for our current dataset selection was to ensure reproducibility and provide precise "ground-truth" labels for anomaly detection.

However, our results provide strong indicators for enterprise scalability. The achieved throughput of 52K events/s per processing unit suggests that LogSqueeze can handle the peak traffic of high-density nodes in real-time. In a massivescale enterprise deployment, the system is designed to operate in a distributed agent-collector architecture, where localized compression (Section IV.B) minimizes the bandwidth required for cross-system log transmission. The Global Network Tuple Correlation (Section VI.2) further ensures that causality is preserved across millions of multi-system boundaries. We are currently coordinating with industrial partners to conduct long-term pilot studies in live production environments to comprehensively prove the system's performance in terms of cross-domain scalability and diverse system compatibility.

7. Conclusion

In this paper, we design a real-time log compression method based on spatio-temporal features and multi-stage provenance graph evolution, aiming at solving the problem of efficiently processing and analyzing massive log data in large-scale host environment. LogSqueeze is based on streaming data transmission and storage, and completes online template extraction, time dimension compression (time window aggregation + online differential coding) and spatial dimension compression (online dictionary coding) through a three-stage pipeline, and also introduces an adaptive compression strategy to cope with load change scenarios, and finally generates temporal and spatial joint compressed data. On this basis, a multi-stage provenance graph is constructed to dynamically record the causal relationships among entities, node vectorization embedding and anomaly detection are performed by combining Node2Vec and LSTM, and the long-distance dependency capture capability is enhanced by the root node optimization strategy,

and finally, the complete workflow of the method is described by a real attack case. The experimental results show that the method maintains high detection accuracy while the compression ratio significantly outperforms the baseline methods such as CPR and GS (up to $12.6\times$), and at the same time, there is no significant impact on the anomaly detection performance, which effectively improves the efficiency of real-time monitoring and threat response. Experimental results demonstrate the efficiency and robustness of LogSqueeze. Our future work will focus on transitioning from laboratory benchmarks to large-scale enterprise pilot deployments, specifically evaluating the system's adaptive performance across millions of devices and heterogeneous production environments.

CRedit authorship contribution statement

Tiantian Zhu: Writing – original draft, Validation, Methodology, Funding acquisition, Formal analysis, Data curation, Conceptualization. **Yiqian Yang:** Writing – original draft, Software, Methodology, Investigation, Data curation, Conceptualization. **Zhengqiu Weng:** Writing – review & editing, Validation, Supervision, Software, Methodology, Investigation, Funding acquisition. **Haofei Sun:** Software, Resources, Methodology, Investigation. **Mingjun Ma:** Validation, Software, Formal analysis, Data curation. **Zhizhong Ma:** Writing – review & editing, Supervision, Resources, Investigation. **Suyu Zhang:** Writing – review & editing, Supervision, Resources, Funding acquisition.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Acknowledgment

This work is supported in part by the following grants: National Natural Science Foundation of China under Grant No. 62672447. Zhejiang Provincial Natural Science Foundation of China under Grant No. LQ21F020016. The Fundamental Research Funds for the Provincial Universities of Zhejiang under Grant No. RF-A2023009. Wenzhou Key Scientific and Technological Projects under Grant No. ZG2024007. Wenzhou Basic Scientific Research Projects under Grant No. G20240033. Key Laboratory of Data Intelligence and Governance of Wenzhou City under Grant No. KLDI2025009. Provincial Teaching Reform Project for Graduate Education in Zhejiang Province under Grant No. JGCG2025539. Taizhou Science and Technology Planning Project under Grant No. 25gyb50.

Data availability

Data will be made available on request.

References

- Ahmad, R., Bru, M., Gehani, A., 2018. Streaming provenance compression. In: *Provenance and Annotation of Data and Processes: 7th International Provenance and Annotation Workshop, IPAW 2018, London, UK, July 9-10, 2018, Proceedings*. Springer International Publishing, pp. 236–240.
- Altinisik, E., Deniz, F., Sencar, H.T., 2023. Provg-searcher: a graph representation learning approach for efficient provenance graph search. In: *Proceedings of the 2023 ACM SIGSAC conference on computer and communications security*, pp. 2247–2261.
- Author unknown, 2025. Articles by category. URL: <https://linux-audit.com/categories/>.
- Author unknown, 2025. DTrace FreeBSD Wiki. URL: <https://wiki.freebsd.org/DTrace>.
- Carlini, N., et al., 2022. Quantifying memorization across neural language models. In: *Proceedings of the Eleventh International Conference on Learning Representations*.
- Cheng, Z., Lv, Q., Liang, J., et al., 2024. Kairos: practical intrusion detection and investigation using whole-system provenance. In: *Proceedings of the 2024 IEEE Symposium on Security and Privacy (SP)*, pp. 3533–3551.
- K. Cho et al. "Learning phrase representations using RNN encoder-decoder for statistical machine translation". In: *arXiv preprint arXiv:1406.1078* (2014).
- CrowdStrike, 2025. 2025 Global Threat Report. Accessed: 2025-04-13. <https://www.crowdstrike.com/en-us/global-threat-report/>. Accessed: 2025-04-13.

- Darap3, 2023. Transparent Engagement 3. URL: <https://drive.google.com/drive/folders/1QlbUFWAGq3Hpl8wVdzOdfoZLFxkII4EK> (visited on 08/01/2023).
- Ding, H., Zhai, J., Deng, D., et al., 2023. The case for learned provenance graph storage systems. In: Proceedings of the 32nd USENIX Security Symposium (USENIX Security 23), pp. 3277–3294.
- Dolgorsuren, B., Khan, K.U., Rasel, M.K., et al., 2019. StarZIP: streaming graph compression technique for data archiving. *IEEE Access* 7, 38020–38034.
- Gandhi, V., et al., 2023. Rethinking system audit architectures for high event coverage and synchronous log availability. In: Proceedings of the 32nd USENIX Security Symposium (USENIX Security 23). Anaheim, CA. USENIX Association, pp. 391–408. Aug. ISBN: 9781-939133-37-3. URL: <https://www.usenix.org/conference/usenixsecurity23/presentation/gandhi>.
- Ghani, A., Tariq, D., 2012. SPADE: support for provenance auditing in distributed environments. In: ACM/IFIP/USENIX International Conference on Distributed Systems Platforms and Open Distributed Processing. Middleware'12. ACM/IFIP/USENIX, Berlin, Heidelberg: Springer Berlin Heidelberg, pp. 101–120.
360. 2024 Global Advanced Persistent Threat (APT) Research Report. Research Report. 360, 2025. URL: <https://360.net/research/report/>.
- M. Goyal, K. Tatwawadi, S. Chandak, et al. "Deepzip: lossless data compression using recurrent neural networks". In: *arXiv preprint arXiv:1811.08162* (2018).
- Grover, A., Leskovec, J., 2016. Node2vec: scalable feature learning for networks. In: Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining. ACM, pp. 855–864. <https://doi.org/10.1145/2939672.2939754>. URL.
- Hassan, W.U., Bates, A., Marino, D., 2020. Tactical provenance analysis for endpoint detection and response systems. In: Proceedings of the 2020 IEEE Symposium on Security and Privacy (SP). Berkeley, CA. IEEE, pp. 1172–1189.
- Hassan, W.U., Guo, S., Li, D., et al., 2019. Nodoe: combatting threat alert fatigue with automated provenance triage. In: Network and distributed systems security symposium.
- Hossain, M.N., et al., 2018d. Dependence-preserving data compaction for scalable forensic analysis. In: Proceedings of the 27th USENIX Security Symposium (USENIX Security 18). Baltimore, MD. USENIX Association, pp. 1723–1740. Aug. ISBN: 978-1-939133-04-5. URL: <https://www.usenix.org/conference/usenixsecurity18/presentation/hossain>.
- Hossain, M.N., Milajerdi, S.M., Wang, J., et al., 2017. SLEUTH: realtime attack scenario reconstruction from COTS audit data. In: Proceedings of the 26th USENIX Security Symposium (USENIX Security 17), pp. 487–504.
- Hossain, M.N., Wang, J., Weisse, O., et al., 2018a. Dependence-preserving data compaction for scalable forensic analysis. In: Proceedings of the 27th USENIX security symposium (USENIX Security 18), pp. 1723–1740.
- Hossain, M.N., Wang, J., Weisse, O., et al., 2018b. Dependence-preserving data compaction for scalable forensic analysis. In: Proceedings of the 27th USENIX Security Symposium (USENIX Security 18). USENIX, pp. 1723–1740.
- Hossain, M.N., Wang, J., Weisse, O., et al., 2018c. Dependence-preserving data compaction for scalable forensic analysis. In: Proceedings of the 27th USENIX Security Symposium (USENIX Security 18), pp. 1723–1740.
- Hutchins, E.M., Cloppert, M.J., Amin, R.M., 2011. Intelligence-driven computer network defense informed by analysis of adversary campaigns and intrusion kill chains. In: Leading Issues in Information Warfare & Security Research, 1, p. 80.
- Jamshidi, K., Mahadasa, R., Vora, K., 2020. Peregrine: a pattern-aware graph mining system. In: Proceedings of the Fifteenth European Conference on Computer Systems, pp. 1–16.
- Lee, K.H., Zhang, X., Xu, D., 2013. High accuracy attack provenance via binary-based execution partition. In: NDSS, 16.
- Lee, K.H., Zhang, X., Xu, D., 2013b. Loggc: garbage collecting audit log. In: Proceedings of the 2013 ACM SIGSAC conference on Computer & Communications Security, pp. 1005–1016.
- Lee, K.H., Zhang, X., Xu, D., 2013c. High accuracy attack provenance via binary-based execution partition. In: Proceedings of the Network and Distributed System Security Symposium (NDSS). Internet Society, pp. 1–16.
- Li, Z., Chen, Q.A., Yang, R., et al., 2021. Threat detection and investigation with system-level provenance graphs: a survey. *Comput. Secur.* 106, 102282.
- Ma, S., Zhai, J., Kwon, Y., et al., 2018. Kernel-supported cost-effective audit logging for causality tracking. In: Proceedings of the 2018 USENIX Annual Technical Conference (USENIX ATC 18). USENIX, pp. 241–254.
- Mikolov, T., et al., 2013. Efficient estimation of word representations in vector space. *arXiv preprint arXiv:1301.3781*. <https://arxiv.org/abs/1301.3781>.
- Milajerdi, S.M., Gjomemo, R., Eshete, B., et al., 2019. Holmes: realtime APT detection through correlation of suspicious information flows. In: Proceedings of the 2019 IEEE Symposium on Security and Privacy (SP). San Francisco, CA, USA. IEEE, pp. 1137–1152. IEEE.
- Qianxin. *Cybersecurity threats 2024 annual report*. Annual report. Qianxin, 2025. URL: https://www.qianxin.com/threat/reportdetail?report_id=335.
- Radford, A., Wu, J., Child, R., et al., 2019. Language models are unsupervised multitask learners. In: OpenAI Blog, 1, p. 9.
- Red Canary. *Evaluating endpoint products in a crowded, confusing market*. [DB/OL]. 2024. URL: <https://redcanary.com/blog/evaluating-endpoint-products-in-a-crowded-confusing-market/>.
- Rossi, R.A., Zhou, R., 2018. Graphzip: a clique-based sparse graph compression method. *J. Big Data* 5 (1), 10.
- Shabani, N., Beheshti, A., Farhood, H., et al., 2023. A rule-based approach for mining creative thinking patterns from big educational data. *AppliedMath* 3 (1), 243–267.
- Symantec. *About purging Reports*. [DB/OL]. 2023. URL: <https://support.symantec.com/us/en/article/howto129116.html>.
- Tang, Y., Li, D., Li, Z., et al., 2018a. NodeMerge: template based efficient data reduction for big-data causality analysis. In: Proceedings of the 2018 ACM SIGSAC Conference on Computer and Communications Security, pp. 1324–1337.
- Tang, Y., Li, D., Li, Z., et al., 2018b. NodeMerge: template based efficient data reduction for big-data causality analysis. In: Proceedings of the 2018 ACM SIGSAC Conference on Computer and Communications Security. ACM, pp. 1324–1337.
- Wang, S., Wang, Z., Zhou, T., et al., 2022. Threatrace: detecting and tracing host-based threats in node level through provenance graph learning. *IEEE Trans. Inf. Forensics Secur.* 17, 3972–3987.
- Wang, W., et al., 2025. ProvGoutLiner: a lightweight anomaly detection method based on process behavior features within provenance graphs. *Comput. Secur.* 157, 104589. <https://doi.org/10.1016/j.cose.2025.104589>. ISSN: 0167-4048. URL: <https://www.sciencedirect.com/science/article/pii/S0167404825002780>.
- Xie, Y., Feng, D., Tan, Z., et al., 2012. A hybrid approach for efficient provenance storage. In: Proceedings of the 21st ACM International Conference on Information and Knowledge Management, pp. 1752–1756.
- Xie, Y., Feng, D., Tan, Z., et al., 2012. A hybrid approach for efficient provenance storage. In: Proceedings of the 21st ACM International Conference on Information and Knowledge Management. ACM, pp. 1752–1756.
- Xie, Y., Muniswamy-Reddy, K.K., Feng, D., et al., 2013. Evaluation of a hybrid approach for efficient provenance storage. *ACM Trans. Storage (TOS)* 9 (4), 1–29.
- Xie, Y., Muniswamy-Reddy, K.K., Long, D.D.E., et al., 2011. Compressing provenance graphs. In: Proceedings of the 3rd USENIX Workshop on the Theory and Practice of Provenance (TaPP 11).
- Xie, Y., Muniswamy-Reddy, K.K., Long, D.D.E., et al., 2011b. Compressing provenance graphs. In: Proceedings of the 3rd USENIX Workshop on the Theory and Practice of Provenance (TaPP 11). USENIX.
- Xu, Z., Wu, Z., Li, Z., et al., 2016. High fidelity data reduction for big data security dependency analyses. In: Proceedings of the 2016 ACM SIGSAC Conference on Computer and Communications Security, pp. 504–516.
- Yang, F., et al., 2023. PROGRAPHER: an anomaly detection system based on provenance graph embedding. In: Proceedings of the 32nd USENIX Security Symposium (USENIX Security 23). Anaheim, CA. USENIX Association, pp. 4355–4372. Aug. ISBN: 978-1-939133-37-3. URL: <https://www.usenix.org/conference/usenixsecurity23/presentation/yang-fan>.
- Zhu, T., Wang, J., Ruan, L., et al., 2021. General, efficient, and realtime data compaction strategy for apt forensic analysis. *IEEE Trans. Inf. Forensics Secur.* 16, 3312–3325.